

Take-Home Assignment: Voice-Based Vehicle Search

Build a working voice search demo. Present the architecture. Defend the tradeoffs.

Time budget: ~12–16 focused hours over 5–7 days. We are evaluating judgment, not hours — do not gold-plate.

1. The Task

Build a voice-first assistant that lets a buyer find a used commercial vehicle by talking to it.

The core loop you must demonstrate live:

Buyer speaks a need — *"chhota truck chahiye, 5 lakh ke andar, city delivery ke liye"* (English is acceptable; Hindi/Hinglish is a plus) → transcribe → extract intent and slots (budget, body type, fuel, city, purpose) → search a vehicle catalog with **real filters** → speak back the **top 3 results, each with a reason** grounded in catalog facts.

Catalog: synthesize your own — 100+ listings with at least: make, model, year, price, km driven, fuel, payload/GVW, body type, city, papers-verified flag. A CSV or SQLite file is fine. How you generate it is up to you.

2. What You Need to Build

Seven components. Each may be thin, but all seven must exist and appear in your architecture diagram:

1. **Voice interface** — a minimal web page or CLI with push-to-talk mic capture and audio playback. Zero polish required; it just has to work live.
2. **STT stage** — speech to text, vendor API or open-source model, streaming or file-based.

3. **Understanding stage** — intent + slot extraction (LLM function-calling, or rules — your call): budget, body type, fuel, city, purpose. Slot state must be inspectable at every turn.
4. **Catalog + search layer** — your synthesized catalog in a queryable store, a filter builder that converts slots into **hard filters**, and a ranking of what remains.
5. **Response layer** — composes the top-3 spoken answer strictly from the returned records (your anti-hallucination mechanism lives here), then TTS.
6. **Conversation state** — slot memory across turns, so corrections update one slot and follow-ups know which vehicle “uska” refers to.
7. **Eval harness + latency log** — a script that runs your 10+ test utterances through the real pipeline, checks the executed filters against expected, and prints pass rate; plus per-turn latency timestamps (speech end → first audio back).

Glue them however you like — one process is fine. What we care about is that each seam is identifiable, because §5’s tradeoff questions are asked per component.

3. Hard Requirements

1. **End-to-end voice.** Mic in → STT → understanding → search → TTS out. Any vendors or open-source models; free tiers are fine.
2. **Filters, not vibes.** A stated constraint is a wall: a buyer who said “under 5 lakh” must never hear a 6-lakh truck. Slot extraction must be visible (logged or on screen) so we can verify what the system understood.
3. **Zero invented facts.** Every number the assistant speaks (price, km, payload, year) must come from the catalog. Be ready to show the mechanism that enforces this — a prompt alone is not a mechanism.
4. **Handle the awkward turns.** Zero results (say so, suggest a relaxation), a correction mid-conversation (“*nahi diesel nahi, CNG*” — update one slot, don’t restart), and at least one follow-up question about a returned vehicle.
5. **Measure it.** Report per-turn latency (speech end → first audio back), and build a mini eval set: 10+ test utterances with expected filters, with your pass rate. We will run it.

4. Stretch Goals (optional — pick at most two)

Hinglish code-switching · noisy-audio robustness · streaming partial responses · a re-ranker with explainable weights · cross-turn memory · a ₹/conversation cost estimate for your stack.

5. Explicitly Out of Scope

Auth, payments, bookings, mobile apps, deployment/infra. A localhost demo is fine.

6. The Presentation (20 min + 10 min Q&A)

1. **Live demo** — the loop above, including one correction and one zero-result case.
2. **Architecture diagram** — every box justified: why this component, what it hides, what would replace it.
3. **Three tradeoffs minimum**, each stated as: the decision, the alternative you rejected, and why — e.g. cascaded STT→LLM→TTS vs speech-to-speech; vendor API vs open-source model; search engine choice; latency vs quality vs cost.
4. **The scale question**: what breaks first at 100k conversations/month, and what you would change — in order.

7. Evaluation Rubric (100 points)

Criterion	Points	What we look for
Working end-to-end demo	25	The loop runs live, voice to voice, without hand-holding
Search correctness	20	Filters honored, corrections work, eval-set pass rate
Zero-hallucination discipline	15	An enforcement <i>mechanism</i> , and honesty about its limits
Architecture reasoning & tradeoffs	20	Decisions defended against real alternatives, not narrated
Latency awareness	10	Measured numbers, knows where the time goes
Code quality & presentation clarity	10	Readable code, a presentation that answers before we ask

7.1 HOW EACH CRITERION IS SCORED

Working end-to-end demo (25)

- 22–25: the full loop runs live, voice to voice, first attempt — including the correction and zero-result cases.
- 15–21: runs with one retry, or one stage falls back (e.g. typed input when the mic misbehaves) but recovers.
- 8–14: only parts work live; pre-recorded audio or a broken stage stitched by hand.
- 0–7: no working path from spoken input to spoken answer.

Search correctness (20)

- 17–20: every stated constraint honored in every demo turn, corrections update exactly one slot, eval-set pass rate $\geq 90\%$.
- 12–16: constraints honored; eval pass rate 70–90% or a clumsy (but not wrong) correction flow.
- 6–11: an occasional constraint slip caught and admitted, or eval pass rate $< 70\%$.

- 0–5: a budget/fuel/city violation in the live demo.

Zero-hallucination discipline (15)

- 13–15: a code-level mechanism (output validator, structured generation from records, or equivalent) — and the candidate can say precisely what it does *not* catch.
- 8–12: grounding via structured prompt + response template, with honest limits.
- 3–7: prompt instructions only, untested.
- 0–2: the assistant invented a number or a vehicle during the demo.

Architecture reasoning & tradeoffs (20)

- 17–20: ≥ 3 decisions each with a genuinely considered rejected alternative; costs and latency quantified; the scale answer names a real first bottleneck (serial LLM calls, session concurrency, vendor rate limits — not “add more servers”).
- 12–16: sound reasoning; some alternatives feel generic or post-hoc.
- 6–11: can describe what was built but not why it beat anything else.
- 0–5: cannot explain their own design under Q&A.

Latency awareness (10)

- 9–10: per-stage numbers (STT / understanding / search / TTS), knows the dominant stage, tried at least one mitigation.
- 5–8: one end-to-end number, no breakdown.
- 0–4: unmeasured, or numbers that don’t survive a live check.

Code quality & presentation clarity (10)

- 9–10: runs from the README in ≤ 10 minutes; the seven components are findable in the code; the presentation answers questions before we ask them.
- 5–8: minor friction in setup or structure.
- 0–4: we couldn’t run it, or couldn’t follow the walkthrough.

7.2 ROLE BARS — SAME RUBRIC, DIFFERENT WEIGHTS

- **Interns:** ≥ 60 with a live working demo is a strong result. We weight the demo, the eval harness, and Q&A curiosity (do they know *why* their system fails when it fails?). Deep tradeoff analysis is a bonus, not an expectation.
- **Tech lead:** ≥ 80 expected, with **$\geq 17/20$ on architecture & tradeoffs as a hard requirement** — a flawless demo with weak tradeoff reasoning does not pass at this level. Q&A will push on failure modes: what happens on noisy audio, on a vendor outage mid-conversation, at 200 concurrent sessions, and what the ₹-per-conversation of their stack is.

7.3 AUTOMATIC CAPS (ANY ONE OF THESE CAPS THE TOTAL AT 40)

- A stated constraint violated in the live demo *and not noticed by the candidate*.
- An invented vehicle or number presented as fact with no detection mechanism.
- Inability to explain code they submitted.

7.4 WHAT Q&A WILL PROBE (SHARED WITH CANDIDATES DELIBERATELY)

1. "Your STT misheard *Ace* as *S*. What happens downstream, and where would you catch it?"
2. "Why did you rank these three first? Show me the reason in data, not in prose."
3. "Which component would you replace first in production, and what's the interface that makes that cheap?"
4. "Your latency is X ms. Where does most of it go, and what's the first 200 ms you'd win back?"

We share these in advance because knowing the questions doesn't help unless the system was built well — which is the point.

8. Rules

- Any language or framework. AI coding assistants are allowed and expected — but you must be able to explain every line, and Q&A will test that.
- Cite anything you borrowed (repos, tutorials, models).

- Submit 24h before presenting: repo link with a README that gets us running in ≤ 10 minutes, your eval set + results, and slides (PDF).

Questions during the assignment are welcome and cost nothing. Wrong assumptions silently baked in cost points.